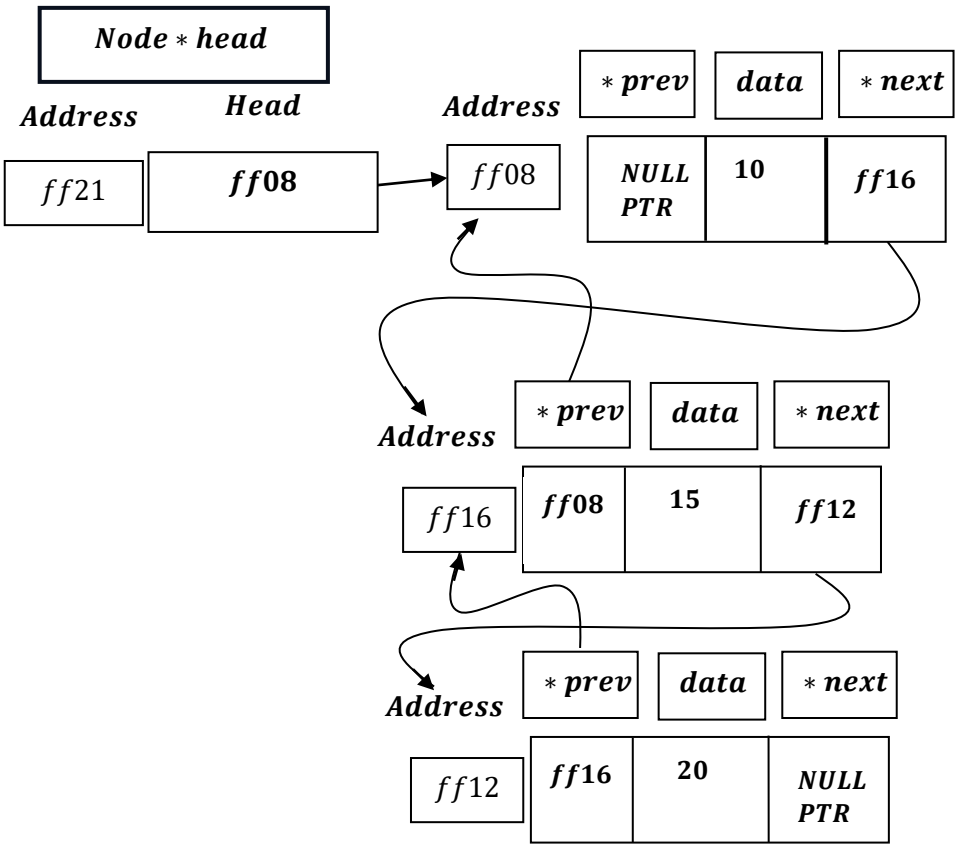
Doubly Linked List – Delete From Beginning And Time Complexity.

Program :

```
void deleteFromBeginning()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    Node *temp = head;
    head = head->next;
    if (head != nullptr)
    {
        head->prev = nullptr;
    }
    cout << "Deleted " << temp->data << " from the
beginning." << endl;
    temp->next = nullptr;
    free(temp);
    temp = nullptr;
}
…..
case 5:
    deleteFromBeginning();
    break;
```

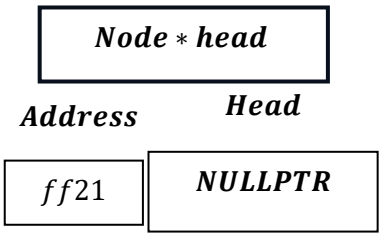
Program Analysis:

Lets consider a Doubly Linked List :



Now, first Line of Code:

```
if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

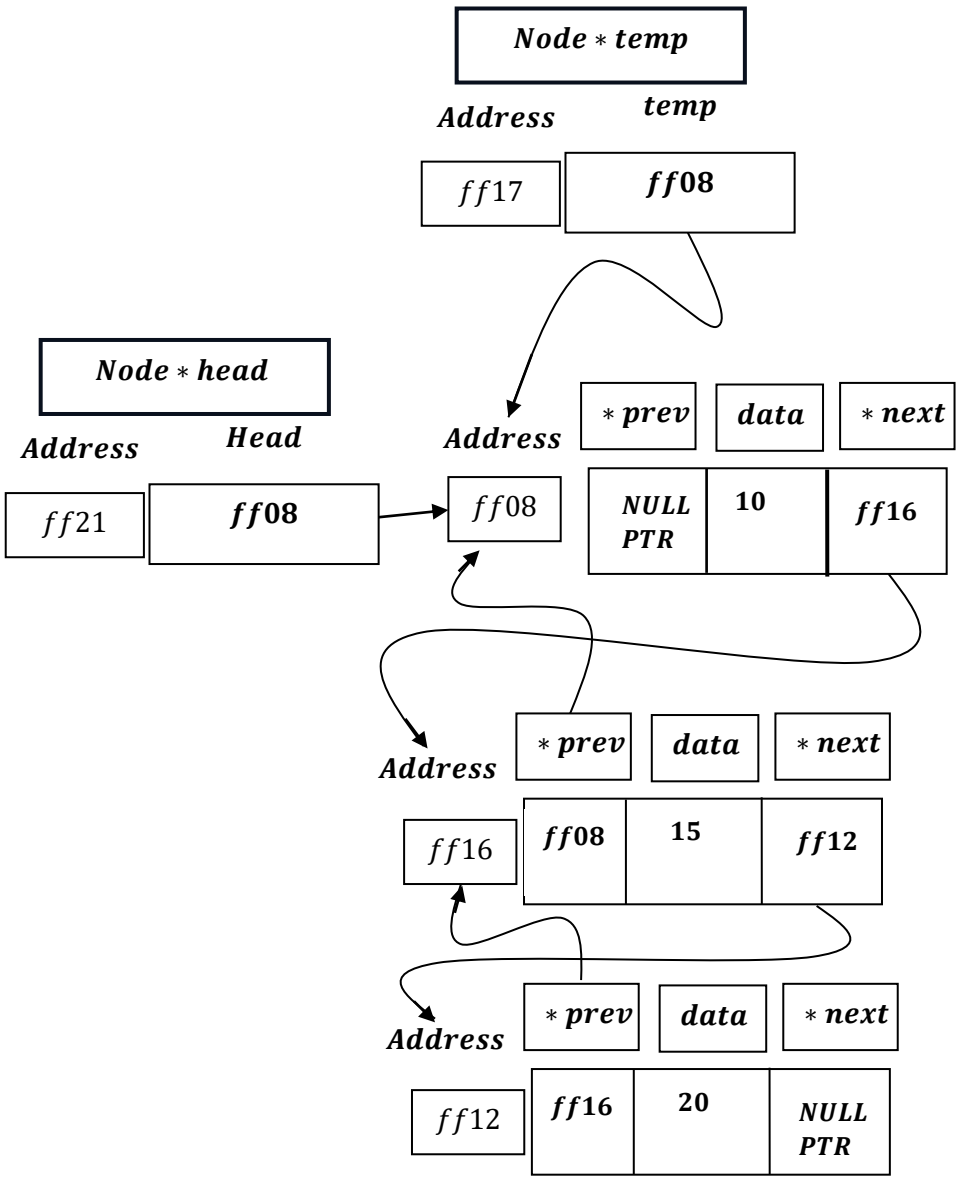


Then, it will print statement : `` List is empty. Nothing to delete.``

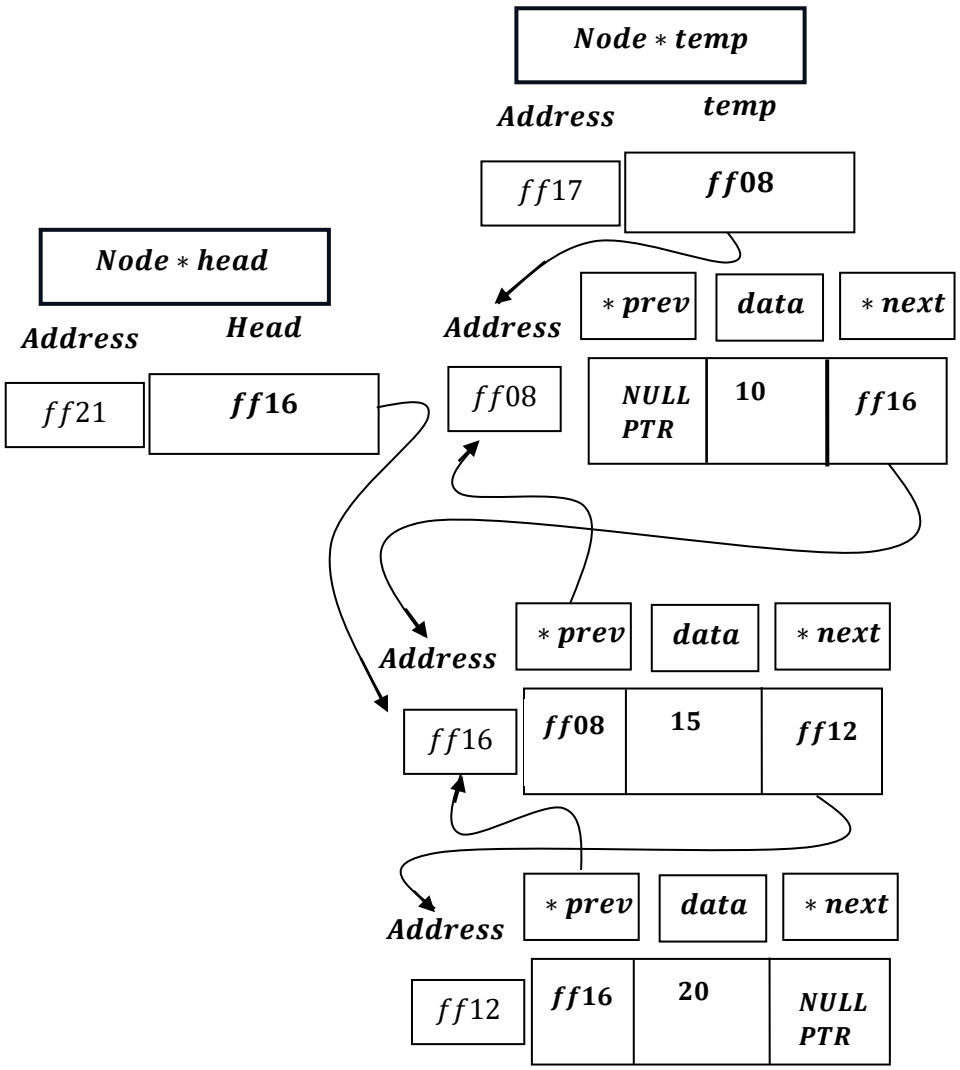
And, return statement will exit from the function.

Next, Line of Code:

```
    Node *temp = head;
```



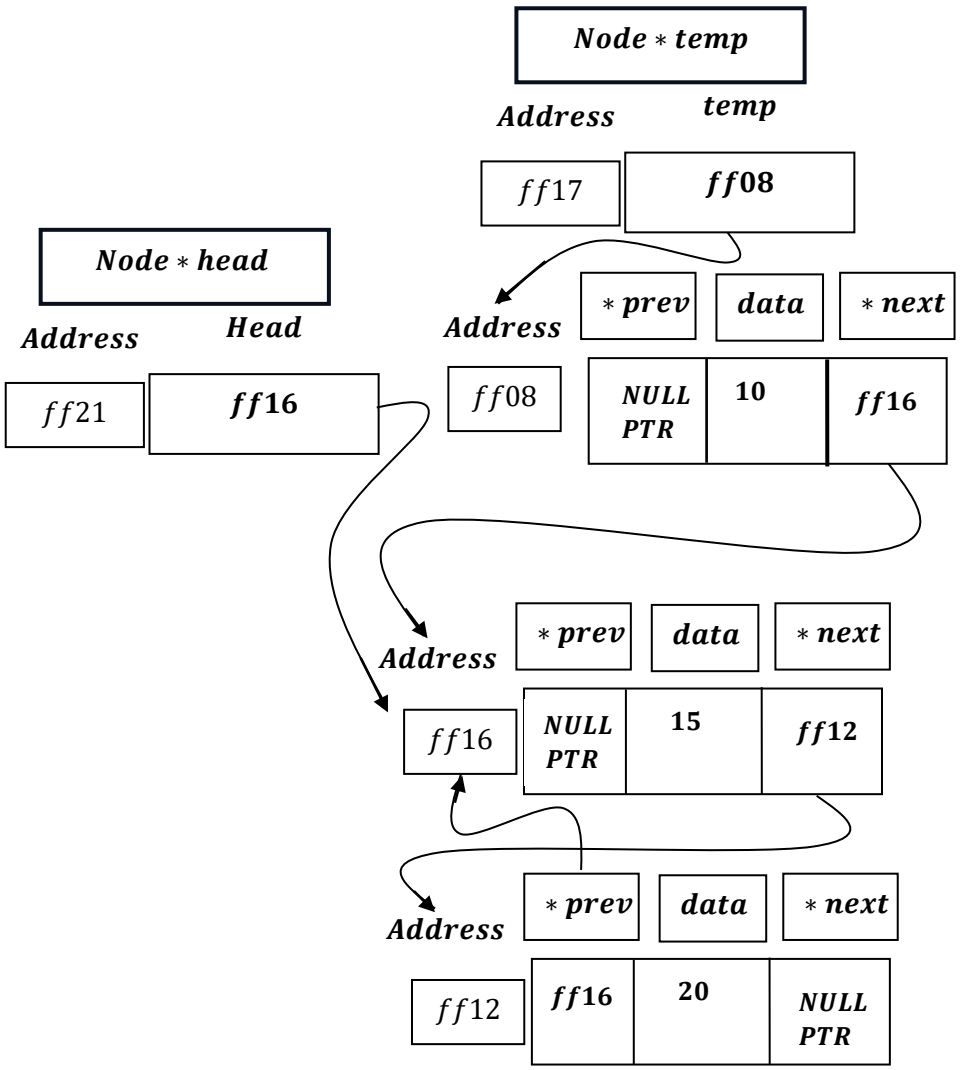

```
    head = head->next;
```



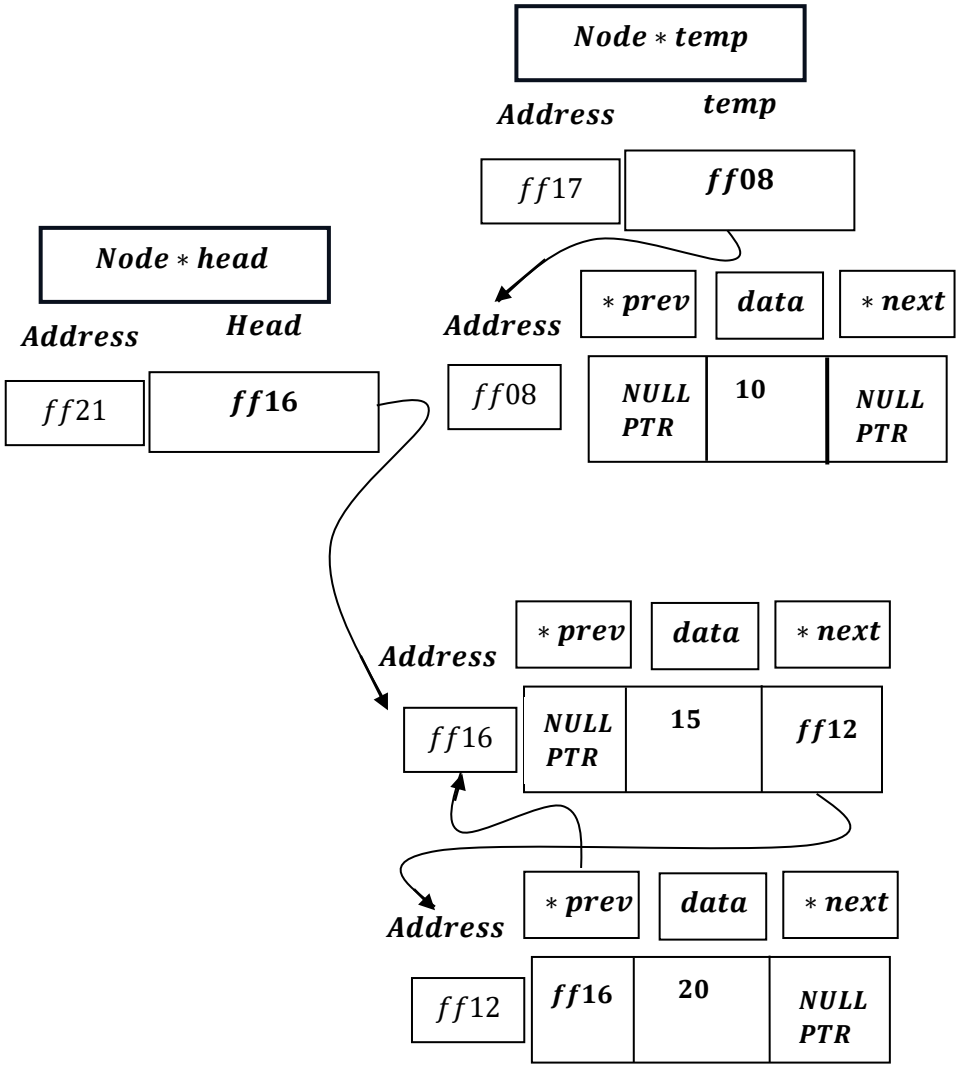
Next, Line of Code:

```
    if (head != nullptr)
    {
        head->prev = nullptr;
    }
```

At this present state , head is not pointing to NULLPTR , Then:

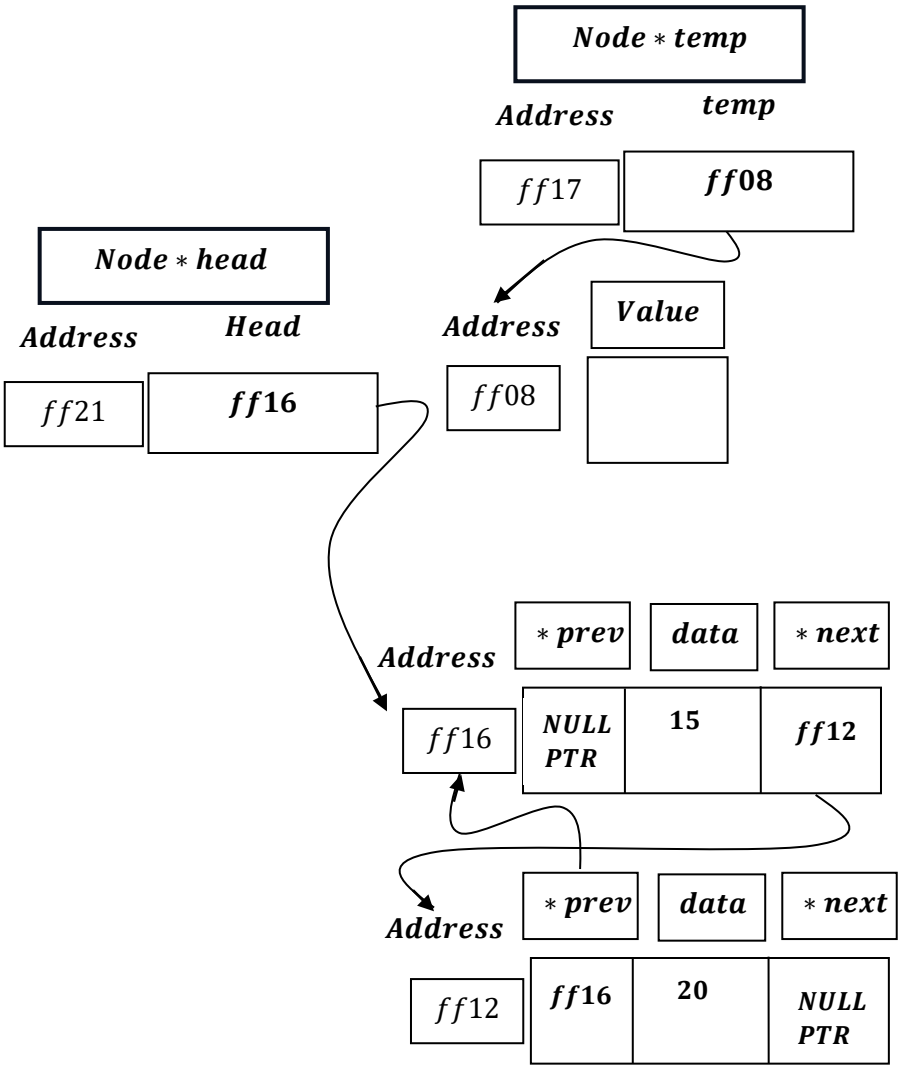



```
    temp->next = nullptr;
```




```
    free(temp);
```

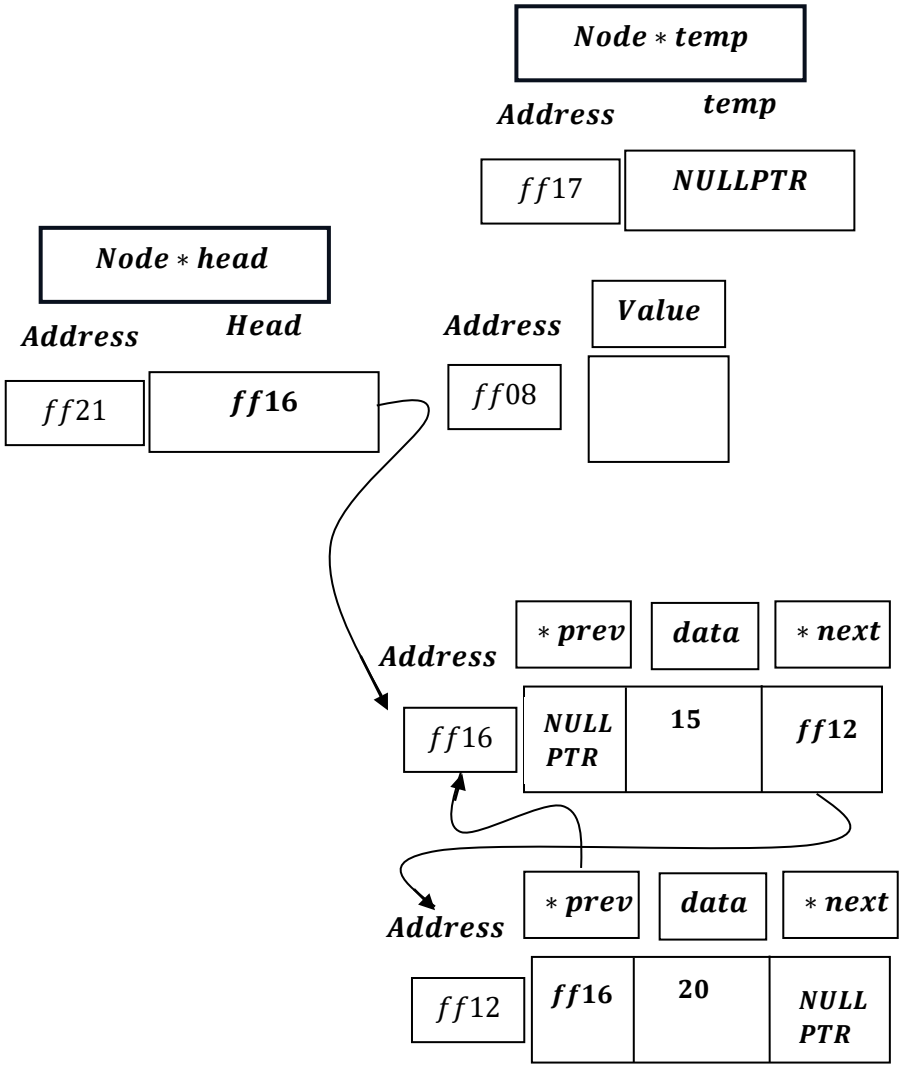
This will free the memory location that temp pointed to:



Now, temp is a dangling pointer. Now, the memory location after freeing , is not a valid memory location , hence the pointer `temp` is now a dangling pointer.

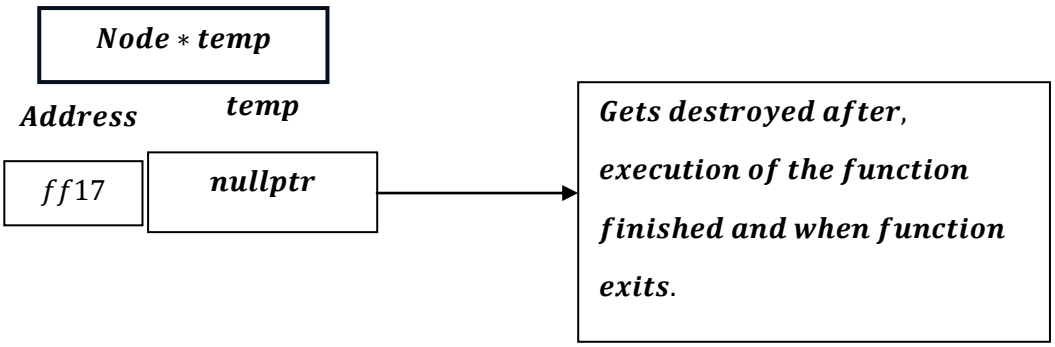
A dangling pointer is a pointer that continues to exist even after the memory it points to has been deallocated or freed. It's called dangling because it is no longer points to a valid object.

```
    temp = nullptr;
```

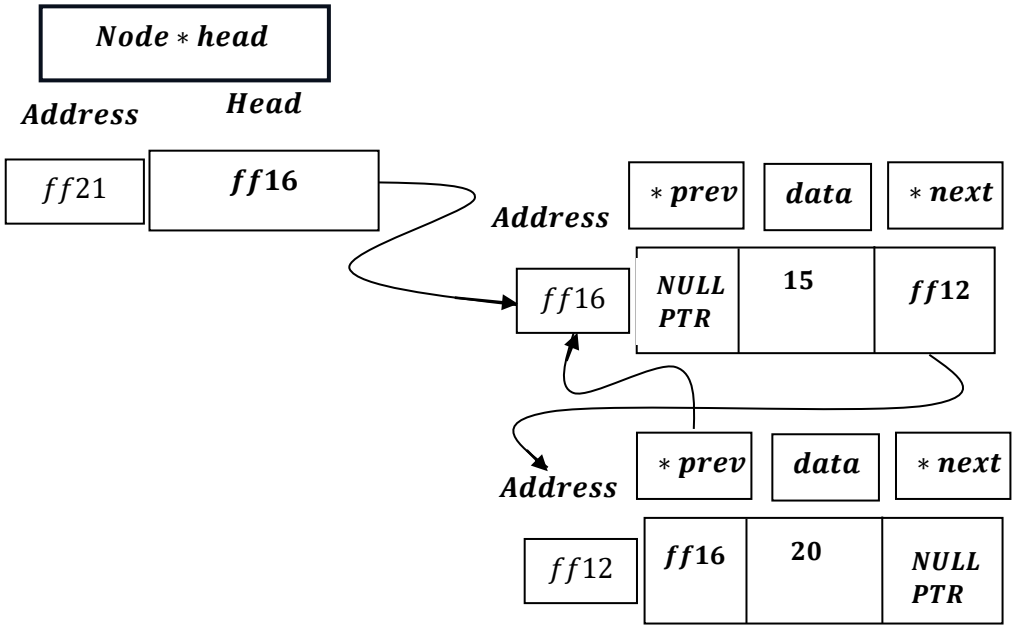


temp = nullptr; → This ensures that any future checks on the pointer (e.g., if (temp != NULLPTR)) will correctly prevent the program from trying to use the invalid memory location.

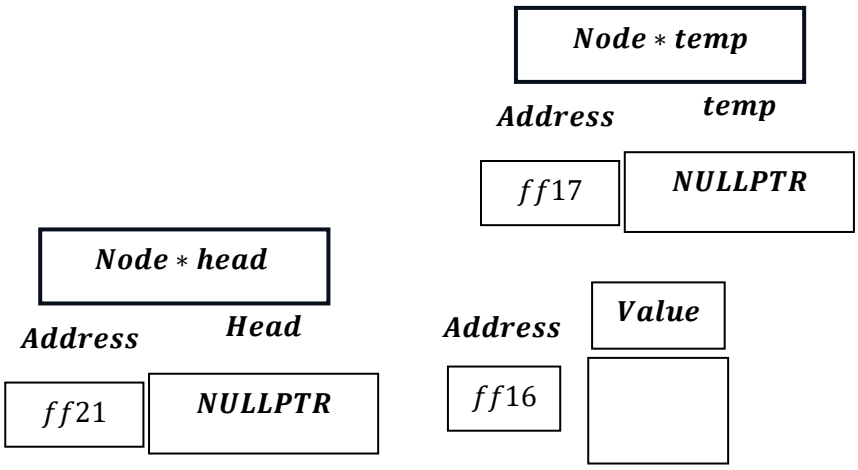
*Node * temp temporary variable, therefore gets destroyed after function execution get finished.*



Hence , we are left with:



Note: −**After , deleting from beginning, List can become Empty**.



Then again :

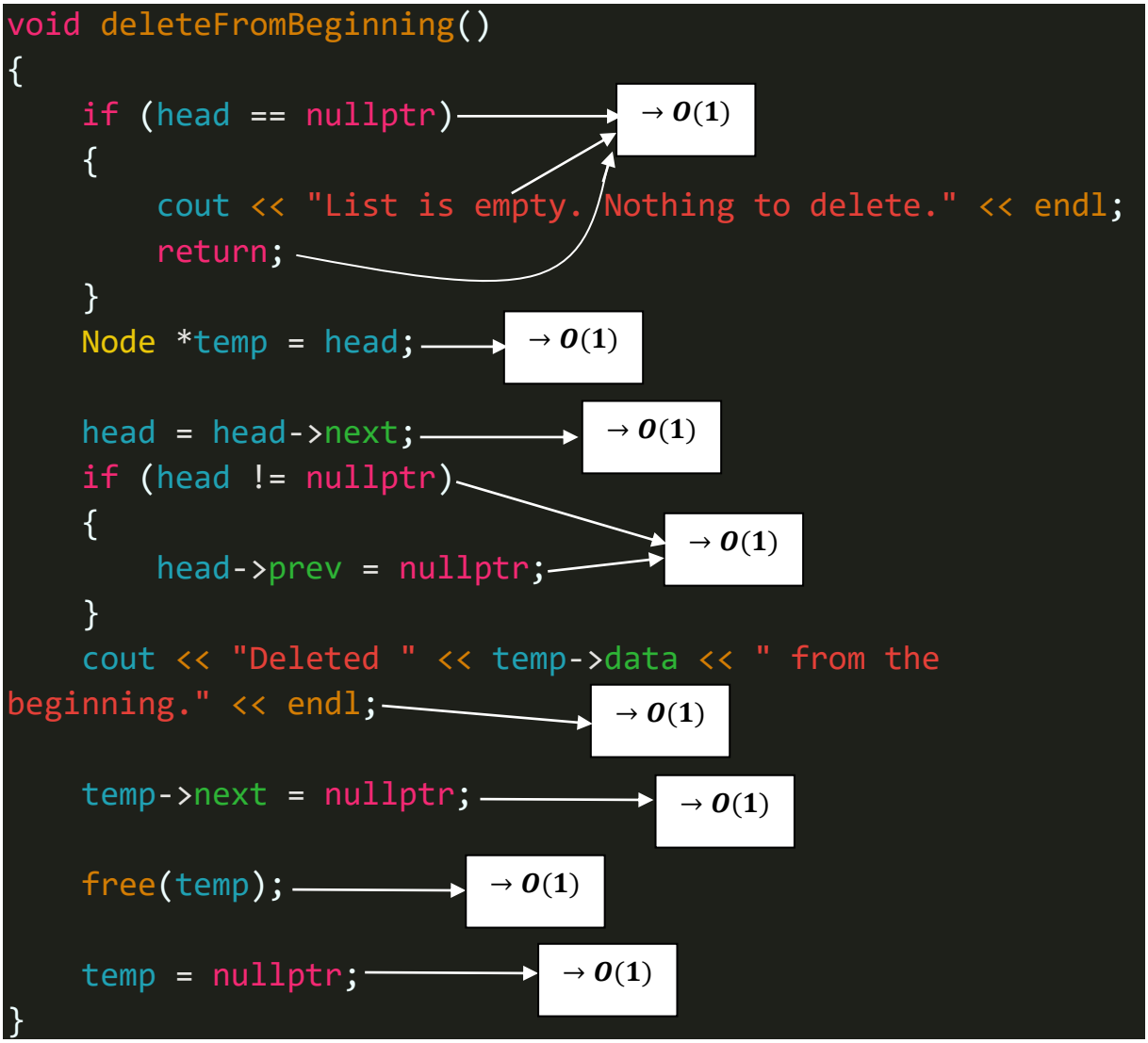
```
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
```

will run and return statement will make exit from the function.

Doubly Linked List – Delete From Beginning [Working Summary]:

1. **Handles Empty List:** It first checks if the list is empty (head == nullptr). If so, it prints a message and stops.
2. **Saves the Head Node:** It creates a temporary pointer, temp, to keep track of the current head node, which is the one to be deleted.
3. **Advances the Head:** The list's official head pointer is moved to the next node in the sequence (head = head->next). This second node is now the new front of the list.
4. **Updates New Head's Link:** It checks if the new head exists (i.e., the list had more than one node). If it does, its prev (previous) pointer is set to nullptr, correctly marking it as the first node.
5. ***The `next` pointer of the old node is nullified as a good practice.***
6. **Deallocates Memory:** The function then frees the memory occupied by the original head node (stored in temp).
7. **Prevents Dangling Pointer:** As a final safety measure, it sets the temp pointer to nullptr to prevent it from pointing to the now-released memory.

Time Complexity of Delete From Beginning Function Module



Best Case = Worst Case = Average Case

1. ***Checking if the list is empty***:

   if $(head == nullptr)$

   – ***This is a simple pointer comparison.***

   – ***Time Complexity***: $O(1)$ ***(constant time)***

   – ***Explanation***: ***Regardless of the size of the list, this operation always takes the same.***

2. *If the list is empty*:

cout << "Error: List is empty." << endl;

return;

– Printing an error message and returning.

– Time Complexity: O(1)(constant time).

– Explanation: Printing a fixed string and returning are both constant time operations.

3. *If the list is not empty*:

*Node * temp = head;*

– Assigning the head pointer to a temporary variable.

– Time Complexity: O(1)(constant time).

– Explanation: This is a simple pointer assignment, which always takes the same amount of time.

4. *Updating the head pointer*:

head = head → next;

– Moving the head pointer to the next node.

– Time Complexity: O(1) (constant time)

– Explanation: This is another pointer assignment, which is a constant time operation.

5. *Checking List is not Empty*

if (head != nullptr)

– This is a simple pointer comparison.

– Time Complexity: O(1) (constant time)

– Explanation: Regardless of the size of the list, this operation always takes the same.

6. ***If the list is not empty***:

head → prev = nullptr;

− Sets prev pointer of memory pointedby head to NULLPTR .

− Time Complexity: ***O*(1) (*constant time*)**

− Explanation: ***Regardless of the size of the list, this operation always takes the same.***

5. ***Printing the deleted node's data***:

cout << "Deleted " << ***temp → data ≪ "from the beginning" ≪ endl***;

− Outputting the data of the node being deleted.

− Time Complexity: ***O*(1) (*constant time*)**

− Explanation: ***Printing a single value is a constant time operation, regardless of the size of the list.***

6. ***Clearing the next pointer of the deleted node***:

temp−> next = nullptr;

− Setting the next pointer of the deleted node to null.

− Time Complexity: ***O*(1)(*constant time*).**

− Explanation: ***This is a simple pointer assignment, which is always constant time.***

7. ***Freeing the memory***:

***free*(*temp*)**;

− Deallocating the memory of the deleted node.

− Time Complexity: ***O*(1) (*constant time*)**

− Explanation: ***Freeing a single node is a constant time operation in most memory management systems.***

8. ***Nullifying the temporary pointer***:

$temp = nullptr;$

– ***Setting the temporary pointer to null.***

– ***Time Complexity***: $O(1)$ ($constant\ time$)

– ***Explanation***: ***This is a simple pointer assignment, which is always constant time.***

Overall Time Complexity:

$O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1).$

– ***Every operation in this function is*** $O(1)$.

– ***When we combine constant time operations, the result is still constant time.***

– ***Therefore, the overall time complexity of `deleteFromBeginning` is*** $O(1)$.

Conclusion:

The `deleteFromBeginning` function has a time complexity of $O(1)$, ***which means it performs in constant time regardless of the size of the linked list. This is because it only operates on the head of the list and does not need to traverse the entire list.***

Note , by relation: $\Omega \leq \Theta \leq O$, ***we get***:

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Rightarrow c_1 \times 1 \leq 1 \leq c_2 \times 1$

Doubly Linked List	Cases	Time Complexity
Deletion From Beginning	Worst Case = Best Case [Hence no true Average Case[As we delete only from first and hence no random selection].] Hence , we can say, Worst Case = Best Case = Average Case.	$\Omega(1) = \Theta(1) = O(1)$.

1. *Best Case*: *The list is empty. The function checks if the head is `nullptr` and returns immediately. This is an* $O(1)$ *operation.*

2. *Worst Case*: *The list is not empty. The function deletes the first node and updates the head pointer. This involves a few pointer manipulations and is also an* $O(1)$ *operation.*

Since the function's behavior does not depend on the size of the list and always performs a constant amount of work, there is no variation in its time complexity. Therefore, there is no distinct average case; the time complexity remains $O(1)$ *in all scenarios.*

****